

HTTP 协议

peanutixx

2026 年 4 月 3 日

Contents

1 URI: 统一资源标识符	1
1.1 URI 的分类	2
1.2 URI 的组成	2
1.3 URL 编码（了解）	3
1.3.1 为什么需要 URL 编码?	4
1.3.2 URL 编码	4
1.3.3 一个完整的示例	5
2 HTTP 报文	5
2.1 HTTP 报文的通用结构	5
2.2 HTTP 请求报文	6
2.3 HTTP 响应报文	8
2.4 关键技术点	10
2.5 请求方法	10
2.5.1 关键特征：安全、幂等、可缓存	10
2.5.2 常见的请求方法	11
2.5.3 小结	16
2.6 响应状态码	16
3 Restful API	19
3.1 资源	19
3.2 表现性	19
3.3 状态转移	19
3.4 示例	20

超文本传输协议（*HTTP*）是一个用于传输超媒体文档（例如 HTML）的应用层协议。它是为 Web 浏览器与 Web 服务器之间的通信而设计的，但也可以用于其他目的。HTTP 遵循经典的**客户端—服务端模型**，客户端打开一个连接以发出请求，然后等待直到收到服务器端响应。HTTP 是一个**无状态协议**，这意味着服务器不会在两个请求之间保留任何数据（状态）。

在深入学习 HTTP 协议之前，我们有必要先去学习 URI 这个概念。

1 URI: 统一资源标识符

统一资源标识符 (*Uniform Resource Identifier, URI*) 是用来标识 Web 上的“资源”。URI 通常用作 HTTP 请求的目标，代表请求资源的位置。URI 中最常见的类型是 URL(统一资源定位符)，也就是我们俗称的网址。

简单来说，URI 是用来标识某一资源的字符串。它的作用就是给互联网上的资源一个唯一的“名字”或“地址”，以便人们能够找到它或者引用它。“资源”可以是任何东西，比如：

- 网络上的：一个网页、一张图片、一个视频、一个 API 接口。

- 本地的：电脑里的某个文件。
- 抽象的：一个电子邮件地址、一本书的 ISBN 编号、一个人。

URI

URI 是 Web 架构的基石之一，它让整个互联网的资源有了统一、规范的命名方式。

1.1 URI 的分类

URI 是一个通用的概念，它又可以分为两大类：URL 和 URN：

- *URL (Uniform Resource Locator, 统一资源定位符)*。URL 不仅标识一个资源，还提供了找到该资源的方法，即资源在**网络中的位置**。URL 就像一个人的家庭住址，通过这个地址，你不仅知道这个人是谁（标识），还能找到他（定位）。

我们平时说的“网址”基本上都是 URL。比如，<https://www.google.com> 就是一个 URL。

- *URN (Uniform Resource Name, 统一资源名称)*。URN 仅标识一个资源，而不说明它在哪、如何访问。URN 就像一个人的**身份证号码**。身份证号码可以唯一地标识这个人，但你无法通过身份证号码直接找到他。

URN 的格式为 `urn:<namespace>:<identifier>`，比如：

- `urn:isbn:0451450523`：标识一本国际标准书号 (ISBN) 为 0451450523 的书，但这串字符并不能告诉你如何下载这本书。
- `urn:uuid:6e8bc430-9c3a-11d9-9669-0800200c9a66`：一个全局唯一的标识符。

1.2 URI 的组成

一个 URI 通常由三部分组成：方案 (*Scheme*)、主机信息 (*Authority*) 和路径 (*Path*)。但一个完整的 URI 可以拥有最多五个部分 (参见 [RFC 3968](#))，下面用最常见的 URL 作为例子来解释：

`https://john.doe@www.example.com:8080/docs/file1.html?name=John#section1`

我们把这个地址拆开来看：

1. 方案 (*Scheme*) 为 `https://`。它是用来告诉浏览器或客户端应该使用什么**协议**去访问资源。常见的协议有 `http://`、`https://`、`ftp://`、`file://`、`mailto:` 等。
2. 主机信息 (*Authority*) 为 `john.doe@www.example.com:8080`。用来标识托管资源的服务器位置。它又可以分为 3 个子部分：
 - 用户信息 (可选)：`john.doe`。可以包含用户名和密码，用于身份验证。由于安全原因，在现代 Web 中，直接在 URI 中嵌入用户信息已不常见。
 - 主机：`www.example.com`。服务器的域名或 IP 地址。这是必填的。
 - 端口 (可选)：8080。服务器监听的网络端口。HTTP 默认是 80，HTTPS 默认是 443。如果使用的是默认端口，可以省略。
3. 路径 (*Path*) 为 `/docs/file1.html`。指向服务器上的特定资源。它的结构和文件系统中的路径很相似，用 `/` 来分隔不同的层级。
4. 查询参数 (*Query*) 为 `?name=John`。向服务器传递额外的参数，通常用于执行搜索、过滤结果或从动态页面获取特定内容。它以 `?` 开头，参数以 `key=value` 的键值对形式存在，多个参数之间用 `&` 分隔 (例如 `?name=John&age=25`)。
5. 片段 (*Fragment*) 为 `#section1`。指向资源内部的一个特定部分。例如，在一个很长的 HTML 文档中，它可以指示浏览器直接滚动到 `id="section1"` 的锚点位置。片段不会被发送给服务器，仅在客户端 (如浏览器) 内部使用。

URI 的组成

<scheme>://<authority><path>?<query>#<fragment>

其中 <authority> 又可以细分为:

<username>:<password>@<host>:<port>

示例: 解析 URI

接下来, 我们使用 wfrest 框架来解析 URI, 完整代码见 examples/01_parse_uri.cc。

01_parse_uri.cc

```
15 int main()
16 {
17     // 1. 使用默认参数, 创建 HTTP 服务器
18     HttpServer server;
19
20     // 2. 注册路由
21     server.GET("/*", [](const HttpReq* req, HttpResp* resp) {
22         // 获取请求 URI
23         cout << "uri: " << req->get_request_uri() << endl;
24         // 解析路径
25         cout << "full_path: " << req->full_path() << endl; // 路由路径
26         cout << "match_path: " << req->match_path() << endl; // *号匹配的路径段
27         cout << "current_path: " << req->current_path() << endl; // 用户传过来的路径
28
29         // 解析查询参数
30         const map<string, string>& querys = req->query_list();
31         for (const auto& [name, value] : querys) { // 结构化绑定: C++17
32             cout << name << ": " << value << endl;
33         }
34
35         // 片段: 客户端内部使用, 不会发送给服务器
36     });
37
38     if (server.start(9527) == 0) { // 监听在通配符地址, 端口为 9527
39         getchar(); // 按任意键退出
40         server.stop(); // 优雅退出
41     } else {
42         cerr << "ERROR: Server start failed!" << endl;
43         exit(1);
44     }
45 }
```

1.3 URL 编码 (了解)

我们在浏览器中经常可以看到带 % 的 URL, 比如:

<https://www.google.com/search?q=C%2B%2B%20%26%20C%23%20%E5%8C%BA%E5%88%AB%3F>

在上面这个例子中, 我们就使用了 URL 编码 (URL Encoding), 也称作百分号编码。简单来说, URL 编码是一种将 URL 中的不安全字符、特殊字符或非 ASCII 字符, 转换为一种可以通过网络安全传输的格式的机制。它的核心原理是: 将一个字节用 % 和两个十六进制字符表示。

1.3.1 为什么需要 URL 编码?

URL 设计之初，只允许使用 ASCII 字符集中的一个子集，并且其中某些字符还有特殊的含义。随着互联网的发展，我们需要在 URL 中传输各种各样的数据，就会遇到下面这些问题：

1. 保留字符：URL 使用一些特殊字符作为语法分隔符，比如：

- / — 用于分隔路径段。
- ? — 用于分隔查询参数和 URL 的其它部分。
- & — 用于分隔多个查询参数。
- = — 用于分隔查询参数的键和值。
- # — 用于分隔锚点和 URL 的其它部分。

如果你需要在查询参数中包含这些字符，比如你想搜索关键词 `keyword=cats&dogs`，其中 `&` 就会被误认为是查询参数分隔符，从而导致 URL 结构被破坏。

2. 不安全字符：有些字符在 URL 上下文中具有特殊意义或可能引起问题，例如：

- 空格：URL 中不能包含空格，空格通常会被编码为 `%20` 或 `+`。
- 其他诸如：", <, >, \, ^, |, {, } 这些符号，都可能被 Web 服务器误解。

3. 非 ASCII 字符：URL 是基于 ASCII 字符集的，无法直接表示中文、日文、阿拉伯文、表情符号等字符。例如，你想在 URL 中包含搜索词“你好世界”，直接放进去是无效的。

URL 编码就是为了解决这些问题的。它将所有有问题的字符转换成一种“安全”的格式，让它们可以被准确无误地传输和解析。

1.3.2 URL 编码

URL 编码遵循一套非常简单的规则：

- 安全字符：`a-z`, `A-Z`, `0-9` 以及一些特殊字符（如 `-`, `_`, `.`, `~`）被认为是安全的，不需要编码，它们在 URL 中保持原样即可。
- 所有其他字符都需要进行编码。
- 编码格式：一个需要编码的字符，会被转换成一个或多个字节（通常使用 UTF-8 编码），然后将每个字节表示为一个 `%` 和两位十六进制数（通常大写）。

如下所示：

字符	ASCII 值	十六进制	URL 编码
␣ (空格)	32	20	%20
=	61	3D	%3D
&	38	26	%26

对于非 ASCII 字符（如：中文，日文，韩文，emoji 符号等），以“好”字为例，URL 编码的步骤如下：

1. 确定字符编码，现代 Web 标准几乎都使用 UTF-8。
2. 查找“好”字的 UTF-8 编码，它由三个字节组成：`E5`, `A5`, `BD`。
3. 将每个字节前加上 `%`。“好”字的 URL 编码为：`%E5%A5%BD`。

1.3.3 一个完整的示例

假设你想通过 Google 搜索一个编程问题：“C++ & C# 的区别？”。这个搜索词包含了很多不安全字符：+，&，#，空格和中文。如果你直接把搜索词拼在 URL 里：

`https://www.google.com/search?q=C++& C# 区别？`

这个 URL 是无效且错误的。浏览器会自动为你进行 URL 编码，编码后的 URL 是这样子的：

`https://www.google.com/search?q=C%2B%2B%20%26%20C%23%20%E5%8C%BA%E5%88%AB%3F`

解码过程如下：

- `C%2B%2B` — 解码后是 `C++`。
- `%20` — 解码后是 `␣`（空格）。
- `%26` — 解码后是 `&`。
- `%20` — 解码后是 `␣`（空格）。
- `C%23` — 解码后是 `C#`。
- `%20` — 解码后是 `␣`（空格）。
- `%E5%8C%BA%E5%88%AB` — 解码后是“区别”。
- `%3F` — 解码后是 `?`。

当 Google 的服务器接收到这个请求后，会自动进行 URL 解码，还原出原始的搜索词“C++ & C# 的区别？”，然后进行搜索。

URL 编码

URL 编码是互联网通信中一个基础但至关重要的组成部分，它确保了数据在不同系统间传输时的完整性和正确性。

2 HTTP 报文

HTTP 报文是 HTTP 协议进行通信的基本单元。当客户端和服务端进行交互时，数据总是以报文的形式进行传输。我们可以把 HTTP 报文看作是一份遵循特定格式的“信件”或“备忘录”，它清楚地说明了“用户想做什么”以及“服务器的回应是什么”。

HTTP 报文分为两大类：

- 请求报文 (*Request*)：客户端 → 服务器，表明“用户想做什么”。
- 响应报文 (*Response*)：服务器 → 回客户端，表明“服务器的回应是什么”。

2.1 HTTP 报文的通用结构

无论是请求报文还是响应报文，都遵循一个经典的“三部曲”结构：

1. 起始行 (*Start Line*)：描述请求或响应的基本信息（如请求方法、状态码、HTTP 版本）。
2. 首部字段 (*Header*)：使用 **key:value** 的形式，可以包含各种各样的信息，比如：对请求/响应的进一步描述、元数据、客户端信息、服务器信息等。
3. 主体 (*Body*)：包含要发送的具体数据内容。主体既可以是文本数据，如提交的表单数据、服务器返回的 HTML 文件等；也可以是二进制数据，如图片、音视频等。主体是可选的，有些报文是没有主体的。



Figure 1: HTTP 报文格式

注意：在 *HTTP* 协议中，我们是使用回车换行 (*CRLF*, `\r\n`) 作为每一行的结束符。

2.2 HTTP 请求报文

请求报文的格式如 Figure 2 所示：

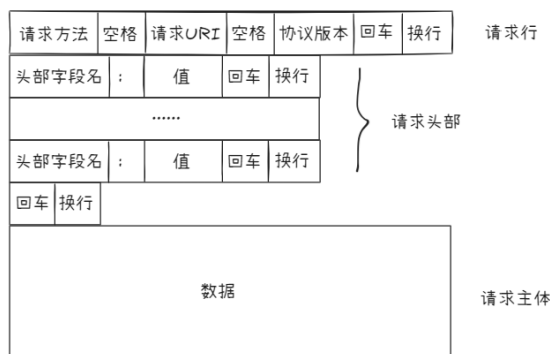


Figure 2: HTTP 请求报文格式

- 请求行 请求报文的第一行称为请求行，由 `<请求方法>`、`<URL>`、`<协议版本>` 字段组成。
 - 请求方法：常用的方法有 `GET`、`POST`、`PUT`、`HEAD`、`DELETE` 等，详见 Section 2.5。
 - URL：通常是一个不含域名的路径（如 `/index.html` 或 `/api/users?page=2`），指明要操作的目标资源。
 - 协议版本：目前常用的版本为 `HTTP/1.1`。

如：`GET /search?q=HTTP HTTP/1.1`

- 请求头部 请求头部用于向服务器传递附加信息。常见的请求头部有：
 - 通用头部：在请求和响应报文中均可使用，但与报文主体（传输的数据）无关的头部字段。如：

`Date: Wed, 18 Oct 2023 10:00:00 GMT` — 报文创建时间。

`Cache-Control: no-cache` — 缓存指令。
 - 请求头部：请求头部仅在请求报文中出现，它们的作用是告诉服务器“我是谁”、“我想要什么”、“我能接受什么”，帮助服务器生成最合适的响应。

`Host: www.example.com` — `HTTP/1.1` 要求必须包含，用来指定服务器的主机和端口。

`User-Agent: Mozilla/5.0 ...` — 发起请求的客户端信息（如：浏览器、操作系统等）。

`Accept: text/html, application/json` — 告知服务器客户端能够接受的 MIME 类型。

`Accept-Language: zh-CN` — 告知服务器客户端偏好的自然语言。

Referer: <https://www.google.com/> — 告知服务器请求是从哪个页面链接过来的。

Authorization: Basic xxxx — 客户端的认证信息。

c) 主体头部: 用于描述报文主体。

Content-Type: application/x-www-form-urlencoded — 主体的 MIME 类型。

Content-Length: 128 — 主体的字节长度。

更多详细信息请参考: [Headers](#)

3. 请求主体 请求主体是 HTTP 请求报文中携带的用户数据, 位于请求头部之后, 通过空行 (CRLF) 隔开。如:

表单

```
1 POST /login HTTP/1.1
2 Host: www.example.com
3 Content-Type: application/x-www-form-urlencoded
4 Content-Length: 27
5
6 username=johndoe&password=123456
```

JSON

```
1 POST /api/users HTTP/1.1
2 Host: api.example.com
3 Content-Type: application/json
4 Content-Length: 45
5
6 {"name": "John Doe", "email": "john@example.com"}
```

GET 方法通常是没有主体的。如果需要传递数据, 得通过 URI 的查询参数 (Query) 进行传递。

示例: 解析 HTTP 请求

接下来, 我们使用 wfrest 框架来解析 HTTP 请求, 完整代码见 `examples/02_parse_request.cc`。

02_parse_request.cc

```
39 server.POST("/*", [] (const HttpReq* req, HttpResp* resp) {
40     // 1. 解析请求行
41     cout << req->get_method() << " "
42         << req->get_request_uri() << " "
43         << req->get_http_version() << "\r\n";
44     // 2. 解析请求头部
45     // wfrest 没有提供遍历 HTTP 头部的方法,
46     // 但我们可以使用 workflow 提供的 HttpHeaderCursor 来遍历。
47     HttpHeaderCursor cursor(req);
48     string name;
49     string value;
50     while (cursor.next(name, value)) {
51         cout << name << ": " << value << "\r\n";
52     }
53     cout << "\r\n";
54     // 3. 解析主体: HTTP 的主体既可以是文本, 也可以是二进制数据
55     cout << req->body() << endl;
56 });
```

2.3 HTTP 响应报文

响应报文的格式如 Figure 3 所示：

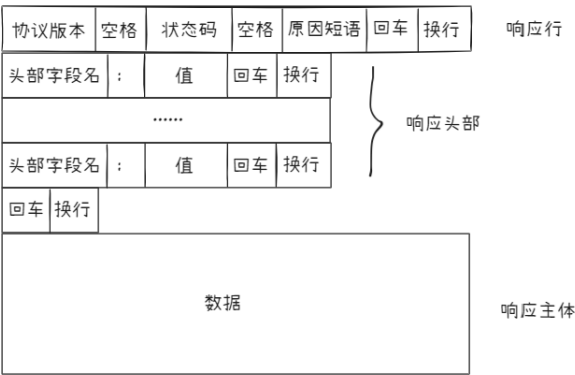


Figure 3: HTTP 响应报文格式

1. 响应行 响应报文的第一行称为响应行，由 <协议版本>、<状态码>、<原因短语> 字段组成。

- a) 协议版本：目前常用的版本为 HTTP/1.1。
- b) 状态码：由三个十进制数字组成，表示响应结果，详见 Section 2.6。
- c) 原因短语：对状态码的简单文字描述，是给愚蠢的人类看的（如 OK、Not Found 等）。

示例：HTTP/1.1 200 OK

2. 响应头部 响应头部是服务器返回给客户端的附加信息。常见的响应头部有：

- a) 通用头部：在请求和响应报文中均可使用，但与报文主体（传输的数据）无关的头部字段。如：
Date: Wed, 18 Oct 2023 10:00:00 GMT — 报文创建时间。
Cache-Control: no-cache — 缓存指令。
- b) 响应头部：响应头部仅在响应报文中出现，它们的作用是告诉客户端“我返回了什么”、“你应该如何处理”、“资源的状态如何”等信息。
Server: Apache/2.4.1 (Unix) — 服务器相关信息。
Location: https://www.example.com/index.html — 重定向时使用（配合 3xx 状态码），告诉客户端接下来该访问哪个网址。
- c) 主体头部：用于描述报文主体。
Content-Type: text/html; charset=utf-8 — 主体的 MIME 类型和字符编码。
Content-Length: 3456 — 主体的字节长度。
Last-Modified: Tue, 17 Oct 2023 15:30:00 GMT — 最后修改时间。
Content-Encoding: gzip — 主体的编码方式（如压缩方式）。

更多详细信息请参考：Headers

3. 响应主体 响应主体是服务器返回给客户端的数据，它代表着服务器的某个资源。响应主体既可以是文本数据(JSON、HTML、XML)，也可以是二进制数据（图片、视频、PDF 等）。

```
1 HTTP/1.1 200 OK
2 Content-Type: text/html; charset=utf-8
3 Content-Length: 135
4
5 <!DOCTYPE html>
```



```

6 <html>
7 <head>
8 <title>示例页面</title>
9 </head>
10 <body>
11 <h1>你好，世界！</h1>
12 </body>
13 </html>

```

示例：解析 HTTP 响应

接下来，我们使用 workflow 来解析 HTTP 响应，完整代码见 `examples/03_parse_response.cc`。

```

18 void http_callback(WFHttpTask* task)
19 {
20     // 1. 检查任务状态 (及早失败原则)
21     // 如果任务执行失败，打印错误信息并退出
22     if (task->get_state() != WFT_STATE_SUCCESS) {
23         cerr << "任务执行失败!" << endl;
24         exit(1);
25     }
26     // 2. 解析响应行
27     HttpResponse* resp = task->get_resp();
28     cout << resp->get_http_version() << " "
29         << resp->get_status_code() << " "
30         << resp->get_reason_phrase() << "\r\n";
31     // 3. 解析响应头部
32     HttpHeadersCursor cursor { resp };
33     string name;
34     string value;
35     while (cursor.next(name, value)) {
36         cout << name << ": " << value << "\r\n";
37     }
38     cout << "\r\n";
39     // 4. 解析响应主体
40     const void* body;
41     size_t size;
42     resp->get_parsed_body(&body, &size); /* 不会复制数据 */
43     cout << "body size: " << size << endl;
44     // 注意：如果响应主体是二进制数据，直接输出会出现乱码
45     // 这里假设响应主体是文本数据
46     cout << static_cast<const char*>(body) << endl;
47 }
48
49 int main()
50 {
51     // 1. 创建一个 HTTP 任务
52     WFHttpTask* task = WFTaskFactory::create_http_task(
53         "http://www.baidu.com", /* request-uri */
54         3, /* redirect-max */
55         3, /* retry_max */

```

```

56     http_callback /* 回调函数：当任务执行完后，才会执行 */
57 );
58 // 2. 设置 HTTP 任务：构建请求内容
59 HttpRequest* req = task->get_req();
60 req->set_method("GET");
61 req->set_request_uri("/");
62 // 3. 将任务交给 workflow 框架异步执行！
63 task->start();
64 // 4. 主线程等待
65 getchar(); // 按任意键终止程序
66 return 0;
67 }

```

2.4 关键技术点

HTTP 报文的设计，有以下几个关键点：

1. 空行 (CRLF)

报文头部和主体之间必须有一个空行。这个空行用来告诉服务器/客户端：“头部到此结束，接下来的内容是主体（如果有的话）。”解析器通过检测连续的 `\r\n\r\n` 来确定头部的结束位置。

2. 长连接

在 HTTP/1.1 中，默认开启长连接 (Connection: keep-alive)。在一次 TCP 连接中可以发送多个 HTTP 报文。在这种情况下，接收方必须通过 Content-Length 或 Transfer-Encoding: chunked 来准确判断每个报文的结束位置，否则无法区分报文的边界。

3. Content-Length 和 Transfer-Encoding

Content-Length — 如果主体的大小是确定的，就用这个头部告诉对方主体的大小。

Transfer-Encoding: chunked — 如果服务器的内容是动态生成的（如实时视频流），无法事先预先知道内容长度，就会使用“分块传输编码”。数据被切割成一块一块发送，每块前面有该块的长度 (Content-Length)，最后跟一个长度为 0 的块表示结束。

小结：HTTP 报文

HTTP 报文是结构化的文本数据（主体部分可以是二进制），它通过 起始行 表明意图，通过 头部 传递元数据，通过 主体 承载具体内容。无论是浏览器还是服务器，都是通过解析这些结构化的文本来理解对方的意思，从而实现万维网上的信息交换。

2.5 请求方法

HTTP 定义了一组 请求方法，告诉服务器客户端想对资源执行什么操作，以及请求成功后的想达到怎样的结果。虽然有些请求方法是名词，比如 OPTIONS，但绝大多数请求方法都是动词，因此请求方法又被称为 *HTTP 动词 (HTTP Verbs)*。

比如，这个请求：`GET /user/1 HTTP/1.1` 表示客户端想获取 id 为 1 的用户信息（回顾：URI 是用来标识资源的）。

每个请求方法都有自己的语义和特征，理解这些语义和特性，有助于设计出更清晰、更符合 HTTP 规范的 Web 服务和 API。

2.5.1 关键特征：安全、幂等、可缓存

在深入每个请求方法之前，理解这三个关键特征是非常重要的，它们可以帮助我们判断“在何种场景下使用哪种请求方法”。

1. 安全 (Safe)：指请求方法不会改变服务器的状态。也就是说，它只是一个“只读”操作。

2. 幂等 (Idempotent): 指一个请求方法执行多次, 其效果等同于执行一次。从数学上讲, 就是 $f(f(x)) = f(x)$ 。
3. 可缓存 (Cacheable): 指该方法的响应是否可以在客户端或代理服务器上被缓存, 以便下次请求相同资源时直接使用。

幂等的重要性

幂等对于网络重试机制至关重要。如果一个请求超时或失败, 客户端可以安全地重试同一个幂等请求, 而不必担心因为多次执行而导致数据不一致 (例如, 多次扣款)。

2.5.2 常见的请求方法

- GET — 获取资源

这是最常用的方法, 用于请求服务器上的资源。比如:

```
1 GET /v1/books/1 HTTP/1.1
2 Host: api.bookstore.com
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
4 Accept: application/json
5 If-None-Match: "abc123" // 条件请求, 如果 ETag 匹配则返回 304
6 Connection: keep-alive
7
```

如果资源未修改, 服务器返回:

```
1 HTTP/1.1 304 Not Modified
2 Date: Mon, 20 Mar 2024 10:30:16 GMT
3 ETag: "abc123"
4 Cache-Control: public, max-age=60
5 Server: nginx/1.18.0
6
```

如果资源发生了修改, 服务器返回:

```
1 HTTP/1.1 200 OK
2 Date: Mon, 20 Mar 2024 10:30:16 GMT
3 Content-Type: application/json; charset=utf-8
4 Content-Length: 98
5 ETag: "def456"
6 Cache-Control: public, max-age=60
7 Server: nginx/1.18.0
8
9 {
10   "id": 1,
11   "title": "三体",
12   "author": "刘慈欣",
13   "price": 68.00,
14   "stock": 100
15 }
```

GET 请求是安全、幂等、可缓存的。它通常用于浏览网页、获取资源详情、搜索列表、加载图片等场景。

- **POST** — 处理数据/提交表单

用于向服务器提交数据（例如，上传表单或文件）。POST 可能会创建新的资源，也可能会更新现有的资源。

```
1 POST /v1/books HTTP/1.1
2 Host: api.bookstore.com
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
4 Content-Type: application/json
5 Content-Length: 86
6 Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
7 Connection: keep-alive
8
9 {
10     "title": "流浪地球",
11     "author": "刘慈欣",
12     "price": 55.00,
13     "stock": 50
14 }
```

创建资源成功，服务器返回：

```
1 HTTP/1.1 201 Created
2 Date: Mon, 20 Mar 2024 10:30:17 GMT
3 Content-Type: application/json; charset=utf-8
4 Content-Length: 115
5 Location: https://api.bookstore.com/v1/books/3
6 Server: nginx/1.18.0
7
8 {
9     "id": 3,
10    "title": "流浪地球",
11    "author": "刘慈欣",
12    "price": 55.00,
13    "stock": 50,
14    "created_at": "2024-03-20T10:30:17Z"
15 }
```

POST 请求是不安全、非幂等、一般不可缓存的。常用于用户登录、提交表单（如：注册，发帖，评论）、创建新资源（如：上传文件/照片）等场景。

POST 请求方法

很多 API 设计会使用 POST 来完成任何非幂等的、复杂的操作。

- **PUT** — 完整替换资源

用于向服务器上传一个全新的资源来完整替换指定 URI 上的已有资源。如果该 URI 上没有资源，一些服务器可能会根据请求创建它。

```
1 PUT /v1/books/3 HTTP/1.1
2 Host: api.bookstore.com
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
4 Content-Type: application/json
```

```
5 Content-Length: 90
6 Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
7 If-Match: "abc123" // 乐观锁, 防止并发修改
8 Connection: keep-alive
9
10 {
11     "title": "流浪地球",
12     "author": "刘慈欣",
13     "price": 66.00,
14     "stock": 30
15 }
```

如果更新成功, 服务器返回的响应报文可能是这样的:

```
1 HTTP/1.1 200 OK
2 Date: Mon, 20 Mar 2024 10:30:18 GMT
3 Content-Type: application/json; charset=utf-8
4 Content-Length: 115
5 ETag: "xyz789"
6 Server: nginx/1.18.0
7
8 {
9     "id": 3,
10    "title": "流浪地球",
11    "author": "刘慈欣",
12    "price": 66.00,
13    "stock": 30,
14    "updated_at": "2024-03-20T10:30:18Z"
15 }
```

如果由于版本冲突, 导致更新失败, 服务器返回的响应报文可能是:

```
1 HTTP/1.1 412 Precondition Failed
2 Date: Mon, 20 Mar 2024 10:30:18 GMT
3 Content-Type: application/json; charset=utf-8
4 Content-Length: 82
5 Server: nginx/1.18.0
6
7 {
8     "error": "Resource has been modified by another user",
9     "code": "PRECONDITION_FAILED"
10 }
```

PUT 请求是不安全、幂等、不可缓存的。常用于更新用户个人资料 (全部信息)、更新文章内容等场景。

PUT V.S. POST

虽然 PUT 和 POST 都是不安全的, 但是 PUT 是幂等的, 而 POST 是非幂等的。PUT 一般用于完整替换资源, 而 POST 一般用于创建资源/提交数据。PUT 请求一般由客户端指定资源的确切 URI (例如 `/users/123`), 而 POST 请求中的 URI 指定的往往是一个集合 (例如: `/users`), 资源确切的 URI 由服务器生成并返回给客户端。

- PATCH — 部分修改资源

用于对资源进行部分修改。它比 PUT 更高效，因为只需要发送需要更改的字段，而不是整个资源。

```
1 PATCH /v1/books/3 HTTP/1.1
2 Host: api.bookstore.com
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
4 Content-Type: application/json-patch+json // JSON PATCH: 支持 add,remove,replace,copy,move 等操作
5 Content-Length: 28
6 Authorization: Bearer eyJhbGciOiJIUzI1NiIs... // JWT 认证
7 Connection: keep-alive
8
9 [
10 { "op": "replace", "path": "/price", "value": 52.80 }
11 ]
```

更新成功后，服务器返回：

```
1 HTTP/1.1 200 OK
2 Date: Mon, 20 Mar 2024 10:30:19 GMT
3 Content-Type: application/json; charset=utf-8
4 Content-Length: 115
5 ETag: "uvw456"
6 Server: nginx/1.18.0
7
8 {
9   "id": 3,
10  "title": "流浪地球",
11  "author": "刘慈欣",
12  "price": 52.80,
13  "stock": 30,
14  "updated_at": "2024-03-20T10:30:19Z"
15 }
```

PATCH 请求是不安全、非幂等、不可缓存的。常用于更新用户个人资料（如邮箱/手机号码）、修改订单的收货地址、文章的阅读量 +1 等场景。

- DELETE — 删除资源

请求服务器删除 URI 上的资源。

```
1 DELETE /v1/books/3 HTTP/1.1
2 Host: api.bookstore.com
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
4 Authorization: Bearer eyJhbGciOiJIUzI1NiIs... // JWT 认证
5 If-Match: "uvw456" // 确保删除的是正确的版本
6 Connection: keep-alive
7
```

删除成功（无返回消息）

```
1 HTTP/1.1 204 No Content
2 Date: Mon, 20 Mar 2024 10:30:20 GMT
3 Server: nginx/1.18.0
```

4

删除成功（有返回消息）

```
1 HTTP/1.1 200 OK
2 Date: Mon, 20 Mar 2024 10:30:20 GMT
3 Content-Type: application/json; charset=utf-8
4 Content-Length: 52
5 Server: nginx/1.18.0
6
7 {
8   "message": "Book successfully deleted",
9   "deleted_id": 3
10 }
```

删除失败，资源不存在：

```
1 HTTP/1.1 404 Not Found
2 Date: Mon, 20 Mar 2024 10:30:21 GMT
3 Content-Type: application/json; charset=utf-8
4 Content-Length: 58
5 Server: nginx/1.18.0
6
7 {
8   "error": "Book not found",
9   "code": "RESOURCE_NOT_FOUND"
10 }
```

DELETE 请求是不安全、幂等、不可缓存的。常用于注销用户、删除帖子等场景。

- HEAD — 获取 GET 请求的响应头

与 GET 请求方法相似，但响应报文中没有主体，服务器只返回与 GET 请求相同的相应行和头部信息。

```
1 HEAD /v1/books/3/download HTTP/1.1
2 Host: api.bookstore.com
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
4 Accept: application/pdf
5 If-Modified-Since: Mon, 20 Mar 2024 10:00:00 GMT
6 Connection: keep-alive
7
```

响应报文：

```
1 HTTP/1.1 200 OK
2 Date: Mon, 20 Mar 2024 10:30:22 GMT
3 Content-Type: application/pdf
4 Content-Length: 10485760
5 Last-Modified: Mon, 20 Mar 2024 10:00:00 GMT
6 ETag: "abc123def456"
7 Accept-Ranges: bytes
8 Cache-Control: public, max-age=3600
9 Server: nginx/1.18.0
```

HEAD 请求是安全、幂等、可缓存的。常用于下面这些场景：

- 1) 检查链接是否有效（响应状态码是否为 200）。
- 2) 检查资源是否过期（通过 Last-Modified 或 ETag 头部）。
- 3) 下载大文件前，先通过 HEAD 请求获取文件的大小（Content-Length）和类型（Content-Type），再决定是否需要下载。

2.5.3 小结

请求方法	作用	安全	幂等	可缓存?
GET	获取资源	✓	✓	✓
HEAD	只获取响应头	✓	✓	✓
POST	处理数据/提交表单	×	×	一般不缓存
PUT	完整替换资源	×	✓	×
PATCH	部分修改资源	×	×	×
DELETE	删除资源	×	✓	×

理解这些请求方法，以及它们的安全性和幂等性，是设计健壮、可维护、符合 RESTful 风格的 Web API 的基础。更多详细信息，请参考 [Methods](#)。

2.6 响应状态码

HTTP 状态码是服务器处理请求后返回的三位数字代码，用于表示请求的结果是成功、失败还是需要进一步处理。它们是 Web 开发中调试和分析问题的重要工具。HTTP 状态码可以分为五大类：

分类	含义
1xx（信息性响应）	服务器已收到请求，正在处理中，客户端可继续或忽略此响应。（很少使用）
2xx（成功）	服务器成功接收、理解并处理了请求。
3xx（重定向）	客户端需要采取进一步操作（如自动跳转）才能完成请求。
4xx（客户端错误）	请求包含语法错误或无法实现等问题，服务器无法处理。
5xx（服务器错误）	服务器在处理请求时发生内部错误或功能未实现。

接下来，我们来看看各类别的一些常见状态码：

1xx: 信息性响应（了解）

- 100 Continue: 服务器已收到请求头，客户端应继续发送请求体。用于优化大请求体的发送流程。
- 101 Switching Protocols: 服务器同意客户端切换应用层协议的请求（如从 HTTP/1.1 升级到 WebSocket）。
- 103 Early Hints: 主要用于在最终响应前预加载资源，服务器先返回一些响应头，让浏览器提前加载关键资源。

2xx: 成功

- 200 OK: 请求成功，请求的资源会随响应返回。
- 201 Created: 请求成功且创建了新资源。新资源的 URI 位在 Location 头部，常用于 POST 或 PUT 请求。
- 202 Accepted: 请求已接受但尚未处理完成，最终可能执行也可能不执行，适用于异步或批处理任务。
- 204 No Content: 服务器成功处理但无内容返回。常用于只需客户端执行操作而不需要更新内容的场景（如删除成功）。浏览器应保持页面视图不变。
- 205 Reset Content: 与 204 类似，但要求浏览器重置当前页面（如表单清空）。
- 206 Partial Content: 返回资源的一部分，表示成功处理了客户端的范围请求（请求中带有 Range 头部）。常用于断点续传或分块下载。

3xx: 重定向 — 这类状态码通常表示资源位置发生改变或需要额外的操作，通常会结合 **Location** 头部使用。

- **301 Moved Permanently:** 永久重定向。资源已永久移动到新 URI，客户端后续请求应使用新 URI。用于网站更换域名等场景。
- **302 Found:** 临时重定向。资源临时位于新 URI，这次应使用新 URI，但后续请求仍使用原 URI。注意：协议规定不能改变请求方法，但实际上许多浏览器会将 POST 改为 GET。
- **303 See Other:** 要求客户端必须使用 GET 方法请求另一个 URI 获取资源，常用于 POST/PUT 后指引客户端跳转到一个确认页面。
- **304 Not Modified:** 表示客户端请求的资源未发生修改，可使用缓存。
- **307 Temporary Redirect:** 临时重定向。与 302 类似，但严令禁止改变请求方法，要求重定向后的请求必须使用原来的请求方法和请求主体（如果有的话）。

示例：重定向

接下来，我们使用 wfrest 来演示重定向，完整代码见 `examples/04_redirect.cc`。

```
                                04_redirect.cc
16 int main()
17 {
18     HttpServer server;
19
20     /*
21      *      301: Move Permanently 永久重定向
22      */
23     server.GET("/status/301", [](const HttpReq* req, HttpResp* resp) {
24         // 设置响应状态码
25         resp->set_status(301);
26         // 添加 Location 标头，指向新的 URL
27         resp->add_header_pair("Location", "newpage/301");
28     });
29
30     server.GET("/newpage/301", [](const HttpReq* req, HttpResp* resp) {
31         resp->String("<h1>GET newpage/301</h1>");
32     });
33
34     /*
35      *      303: See Other 跳转到结果页 (POST -> GET)
36      */
37     server.POST("/status/303", [](const HttpReq* req, HttpResp* resp) {
38         resp->set_status(303);
39         resp->add_header_pair("Location", "newpage/303");
40     });
41
42     server.GET("/newpage/303", [](const HttpReq* req, HttpResp* resp) {
43         resp->String("<h1>GET newpage/303</h1>");
44     });
45
46     server.POST("/newpage/303", [](const HttpReq* req, HttpResp* resp) {
47         resp->String("<h1>POST newpage/303</h1>");
48     });
49 }
```

```

50  /*****
51  /*      307: Temporary Redirect 临时重定向 (保持请求方法不变)  */
52  *****/
53  server.GET("/status/307", [](const HttpReq* req, HttpResp* resp) {
54      resp->set_status(307);
55      resp->add_header_pair("Location", "/newpage/307");
56  });
57
58  server.GET("/newpage/307", [](const HttpReq* req, HttpResp* resp) {
59      resp->String("<h1>GET newpage/307</h1>");
60  });
61
62  server.POST("/status/307", [](const HttpReq* req, HttpResp* resp) {
63      resp->set_status(307);
64      resp->add_header_pair("Location", "/newpage/307");
65  });
66
67  server.POST("/newpage/307", [](const HttpReq* req, HttpResp* resp) {
68      resp->String("<h1>POST newpage/307</h1>");
69  });
70
71  if (server.start(8888) == 0) {
72      getchar(); // 按任意键退出
73      server.stop(); // 优雅退出
74  } else {
75      cerr << "Error: server start failed!" << endl;
76      exit(1);
77  }
78 }

```

4xx: 客户端错误 — 这类状态码表示错误由客户端引起，如：语法错误、权限不足或请求的资源不存在等情况。

- 400 Bad Request: 请求语法有误，服务器无法理解。
- 401 Unauthorized: 用户身份未验证，拒绝访问资源。
- 403 Forbidden: 禁止访问，用户无权限访问资源。
- 404 Not Found: 服务器没有找到请求的资源。可能被删除了，也可能本就不存在。
- 405 Method Not Allowed: 请求方法不被允许。如：用 POST 请求方法访问只读资源。响应报文应告知客户端该资源允许的请求方法。
- 408 Request Timeout: 请求超时，服务器等待客户端发送请求的时间过长。
- 409 Conflict: 请求与服务器当前状态冲突，如：编辑资源时，发现服务器资源的版本和客户端的不一致。
- 410 Gone: 资源已被永久删除。
- 413 Content Too Large: 请求实体太大，超出服务器的处理能力。
- 429 Too Many Requests: 客户端在指定的时间内发送了太多的请求，触发限流。
- 451 Unavailable For Legal Reasons: 因法律原因（如政府审查、版权问题）拒绝访问。

5xx: 服务器错误 — 这类状态码表示服务器在处理请求时出错或功能未实现。

- 500 Internal Server Error: 内部服务器错误。服务器遇到意外情况，无法完成请求，是服务器端的通用错误状态码。

- **501 Not Implemented:** 请求的功能未实现。
- **502 Bad Gateway:** 网关错误。当网关或代理服务器，收到上游服务器的无效响应时，可以返回该状态码。
- **503 Service Unavailable:** 服务不可用。服务器因过载或需要维护，而无法处理请求时，可以使用该状态码。响应中通常会包含 **Retry-After** 头部告知客户端服务器的恢复时间。
- **504 Gateway Timeout:** 网关超时。表示网关或代理服务器，未能及时从上游服务器收到响应。
- **505 HTTP Version Not Supported:** 服务器不支持或拒绝支持请求报文中使用的 HTTP 协议版本。

更多详细信息，请参考 [Status Code](#)。

3 Restful API

REST(*Representational State Transfer*, 表述性状态转移)是一种软件架构风格, RESTful API 则是遵循 REST 原则设计的应用程序接口。它利用 HTTP 协议的特性, 以资源为中心进行 API 设计。RESTful API 因其简洁性和通用性, 已然成为 Web 开发中的首选接口设计风格。

理解 REST 架构, 最好的方法就是去理解 *Representational State Transfer* 这个词组到底是什么意思, 它的每一个词代表了什么涵义。如果你把这个名称搞懂了, 也就不难体会 RESTful API 是一种怎样的接口设计了。

3.1 资源

REST 的名字“表现性状态转移”中, 省略了主语“资源”。“表现性”其实指的是“资源”具有外在表现性。

所谓“资源”, 就是 Web 上的一个信息实体, 或者说是网络上的一个具体信息。它可以是一段文本、一张图片、一首歌曲、一种服务, 总之就是一个具体的存在。你可以用一个 URI (统一资源定位符) 指向它, 每种资源对应一个特定的 URI。要获取这个资源, 访问它的 URI 就可以, 因此 URI 就成了资源的地址或标识符。

“上网”, 其实就是操作 Web 上的一些列资源, 或者说和 Web 上的一系列资源互动。

3.2 表现性

“资源”是一种信息实体, 它可以有多种外在表现形式。比如, 文本可以用 txt 格式表现, 也可以用 HTML 格式、XML 格式、JSON 格式表现, 甚至可以采用二进制格式; 图片可以用 JPG 格式表现, 也可以用 PNG 格式表现。现代 API 通常使用 JSON 格式表示服务器的“资源”, 如:

```
1 {  
2     "id": 123,  
3     "name": "peanut",  
4     "email": "peanutixxx@gmail.com"  
5 }
```

URI 只是“资源的标识”, 表示“资源的名字”或“资源所在的位置”。“资源”的具体表现形式, 在 HTTP 协议中, 是通过一些头部字段指定的, 如: **Accept** 和 **Content-Type**。

3.3 状态转移

“上网”, 其实就是用户和服务器上的资源互动的一个过程。在这个互动过程中, 用户可能需要改变服务器上资源的状态, 也就是资源的状态发生了“转移”。

状态如何“转移”呢? 在 REST 架构中, 是通过 HTTP 动词 (请求方法) 来指定的。具体来说就是:

- **GET** — 获取资源。

- HEAD — 获取资源的元数据。
- POST — 用来创建资源（也可用来更新资源）。
- PUT — 用来完整替换资源。
- PATCH — 部分修改资源。
- DELETE — 删除资源。

3.4 示例

接下来，我们来看两个具体示例：

1. 获取用户列表

请求：

```
1 GET /users HTTP/1.1
2 Host: api.example.com
```

响应：

```
1 [
2   {"id": 1, "name": "peanut"},
3   {"id": 2, "name": "xixi"}
4 ]
```

2. 创建新订单

请求：

```
1 POST /orders HTTP/1.1
2 Content-Type: application/json
3
4 {"product": "Laptop", "quantity": 1}
```

响应：

```
1 {"id": 10086, "status": "processing"}
```

REST 架构

REST 是一种软件架构风格，程序员在设计应用程序的时候应遵循 RESTful API 设计的约束。wfrest 框架只是为开发 REST 架构风格的应用程序提供了便利，并不会实现 RESTful API 设计的约束。